Off the Bench

Build an AI-assisted sports technique analyzer with Python, MediaPipe and Gradio

Student tutorial: follow the steps in order and build the project as you read.

Contents

Welcome: what you are building . . . 3

How to use hints . . . 3

How to use this tutorial . . . 3

The roadmap . . . 4

Before you start . . . 5

Set up your project . . . 5

Read the "Setup status" line . . . 5

Optional: AI feedback and privacy . . . 5

The big picture . . . 6

Build it, step by step . . . 7

Step 1: Run the starter and learn how images work . . . 7

Step 2: Choose a sport and a skill level . . . 9

Step 3: Display measurements and suggestions nicely . . . 11

Step 4: Detect landmarks with MediaPipe . . . 13

Step 5: Measure body angles . . . 15

Step 6: Coaching suggestions . . . 19

Step 7: Optional AI feedback . . . 21

Step 8: Optional extensions . . . 23

What you learned . . . 23

Welcome: what you are building

Off the Bench is a web app that looks at a photo of an athlete and gives technique ideas. You will be able to:

1. Choose a **sport** and a **skill level**.
2. Upload a photo of a movement (a jump shot, a kick, a dance pose).
3. Let a computer-vision model find the **body landmarks** (shoulders, elbows, knees, ...) and draw a skeleton on your photo.
4. Measure **joint angles** from those landmarks, such as "the right elbow is about 90 degrees".
5. Turn the measurements into **coaching suggestions** that depend on the sport and level.
6. Optionally ask an **AI model** for extra tips (only if you give consent to send your photo).

The folder you were given is a **scaffold**. The app already launches, but the important functions are simple placeholders marked `TODO`. This tutorial walks you through every `TODO`, in the order you should do them. **It does not hand you the answers.** Instead, for each TODO you will find:

- **Recall**: a reminder of the Python idea you need, with a small example from an unrelated topic (pizza, pets, grades).
- **Your task**: what the code must do, in plain English.
- **Hints**: nudges that get more specific one at a time.
- **Check it**: how to run it and what you should see when it works.

> **WARNING**
> Honesty matters in this project. One photo is one instant. It cannot prove a whole movement is right or wrong, the angles are approximate, and they change a lot with the camera angle. Your app should always say so. You will see this idea again and again.

> **NOTE**
> Instructor note: a separate teacher answer key with full solutions exists in the `answer_key/` folder.

How to use hints

1. **Try first.** Read the Recall and Your task, then spend about **10 minutes** trying on your own. Getting it wrong and fixing it is how you learn.
2. Still stuck? Reveal **Hint 1** only. Try again for a few minutes.
3. Then Hint 2, then Hint 3. Each hint gives away a little more. If you reach the last hint and still need help, ask a mentor.
4. When your code works, run the **Check it** step. If your output differs, read your code line by line and compare it to the expected behavior.

How to use this tutorial

- Keep this PDF open next to your editor. Work through the steps **in order**. Each step builds on the last.
- Whenever you see **Checkpoint**, stop and run the app (or the test shown) before moving on.
- Stuck? Read the **Stuck?** boxes. They list common error messages and what they mean.

- After you edit a Python file, **stop the app (`Ctrl + C`) and run `python app.py` again**.

The roadmap

Step	File	What you build
1	`app.py`	Run the starter, learn how images work, add the color-channel count (TODO 1)
2	`sports.py`	Add a sport, write photo tips and level guidance (TODOs 1-3)
3	`app.py`	Nicer measurement and suggestion display (TODOs 2-3)
4	`pose_analysis.py`	Download the model, detect landmarks, practice TODOs
5	`feedback.py`	Measure angles and classify them (TODOs 1-5)
6	`feedback.py`	Level-aware coaching and drills (TODOs 6-7)
7	`ai_helper.py`	The AI prompt and answer formatting (TODOs 1-3)
8	any	Optional extensions

Before you start

Set up your project

Open a terminal inside the `off_the_bench` folder and run these commands one at a time.

```
python -m venv .venv
source .venv/bin/activate        # Windows: .venv\Scripts\activate
pip install -r requirements.txt
cp .env.example .env             # Windows: copy .env.example .env
python app.py
```

What each line does:

- `python -m venv .venv` creates a private "box" of Python packages just for this project.
- `source .venv/bin/activate` steps into that box. You should see `(.venv)` at the start of your terminal line. **Do this every time you open a new terminal.**
- `pip install -r requirements.txt` installs Gradio (the web page), NumPy (image numbers), Pillow (image files), MediaPipe (pose detection) and python-dotenv (reads your `.env` file).
- `cp .env.example .env` makes your private settings file. You can leave the key blank for now.
- `python app.py` starts the app. Open the address it prints (usually `http://127.0.0.1:7860`).

> **WARNING**
> MediaPipe works with **Python 3.9 to 3.12**. If `pip install` complains about MediaPipe, check your version with `python --version`. The app **still launches without MediaPipe**: it just turns pose detection off and says so in the status line. You can finish Steps 1 to 3 and Steps 5 to 7 using the fake-landmark tests in this tutorial even if MediaPipe will not install.

Read the "Setup status" line

At the top of the page the app prints a line like this:

```
Setup status: MediaPipe is not installed (pip install mediapipe). Pose detection is off. | AI
    feedback is off: no ANTHROPIC_API_KEY found (see .env.example).
```

That line is your dashboard. It has two halves: **pose detection** (MediaPipe plus its model file) and **AI feedback** (an API key). Each half can be on or off independently. When both are on, you will see `Pose detection is ready.` and `AI feedback is available.`

Optional: AI feedback and privacy

AI feedback needs an Anthropic API key (your teacher may give you one) and the extra package: `pip install anthropic`. Put the key in `.env` as `ANTHROPIC_API_KEY=...`.

> **WARNING**
> Never paste your key into a Python file, share it, or commit `.env` to GitHub. Photos of people are personal. This app sends your photo to the AI **only if the user ticks the consent checkbox**. Without it, only the text measurements are sent.

The big picture

```
choose sport -> upload image -> detect landmarks -> measure angles -> coach -> (optional) AI
  sports.py       app.py        pose_analysis.py     feedback.py     feedback.py  ai_helper.py
```

File	Job
`app.py`	The screen, and the `analyze` function that connects every stage
`sports.py`	The list of sports, photo tips, and what to tell each skill level
`pose_analysis.py`	A wrapper around MediaPipe: finds landmarks, draws the skeleton, measures angles. It is kept separate on purpose.
`feedback.py`	**Measurements** (facts from the photo) and **coaching** (opinions based on facts)
`ai_helper.py`	The optional AI prompt and request, with the consent check

Keep one rule in your head all project long: **measurements are facts about the photo; coaching is your interpretation of them. Keep them separate.** `feedback.py` has one function for each.

Build it, step by step

Step 1: Run the starter and learn how images work

File: `app.py` **Function:** `describe_image(image)` **TODO 1**

Goal: understand the flow of the app, what an image looks like to Python, and report the number of color channels of an upload.

1a. Click around

Run `python app.py`. Choose a sport and watch the text under it change (the `show_sport_info` function in `app.py` does that). Upload any photo, **leave "Detect body landmarks" unticked**, and click **Analyze**. You will see "Image received: ... pixels.", a short list of suggestions, and a disclaimer.

1b. How Python sees a picture

Read the comment above `describe_image`. When you drop a file into `gr.Image(type="numpy")`, Gradio hands your function a **NumPy array**: a grid of numbers with the shape **(height, width, 3)**.

- The first two numbers are the rows and columns of pixels.
- The `3` is the number of **color channels**: red, green and blue.
- Every value is between 0 (none of that color) and 255 (full color).

You can see this for yourself in a Python shell:

```
python -c "import numpy as np; a = np.zeros((480, 640, 3), dtype=np.uint8); print(a.shape, a[0,
    0])"
```

```
(480, 640, 3) [0 0 0]
```

A 640 by 480 photo is an array with 480 rows (height comes first!) and 640 columns. The pixel in the top-left corner is three numbers, here `[0 0 0]`, which is black.

> **CONCEPT**
> In `describe_image`, the line `height, width = image.shape[:2]` takes only the first two items of the shape. `[:2]` means "from the start up to, but not including, position 2". That is why height comes before width.

TODO 1: report the color channels

Recall: `.shape` is a tuple, and you pick items out of a tuple by position, counting from 0. A *conditional expression* picks between two values on one line:

```
pizza_slices = (4, 8)
toppings = pizza_slices[1] if len(pizza_slices) == 2 else 0
```

Your task: make the status message also say how many color channels the uploaded image has. A normal color photo has 3. Your code must **not crash** if someone uploads a grayscale image, whose shape has only two numbers.

Hint 1: Which position in the shape tuple holds the channel count? Remember positions start at 0.

Hint 2: A grayscale image has no third number, so asking for it raises an error. NumPy arrays have an `.ndim` attribute that tells you how many numbers are in the shape.

Hint 3: In words: "channels is the third shape number *if* the image has 3 dimensions, otherwise 1". Then put `channels` in the f-string you already return.

Think about it: the starter asks *why might a PNG have 4 channels?* (The extra channel is called **alpha**. Find out what it stores.) Where in `pose_analysis.py` does the code drop it? Search for `:3`.

CHECKPOINT

Restart and analyze a photo. The status box should now mention the pixel size **and** the number of channels (a normal photo says 3). Upload a PNG with transparency and check that you see 4. You can also test from a terminal inside `off_the_bench`: `python -c "import numpy as np; from app import describe_image as d; print(d(np.zeros((480, 640, 4), dtype=np.uint8)))"` should report 640 x 480 and 4 channels.

STUCK?

`IndexError: tuple index out of range` means you asked for a shape position that does not exist (a grayscale image). If nothing changes in the browser, you did not restart the app.

Step 2: Choose a sport and a skill level

File: `sports.py` **TODOs 1, 2, 3**

Goal: add a new sport, write a photo-taking tip for every sport, and describe how feedback should change for each skill level.

Read the data

At the top of `sports.py`, `SPORTS` is a **dictionary** that maps each sport to a **list** of skills. `get_sport_names()` returns just the keys, which is how the dropdown in `app.py` gets its choices:

```
sport = gr.Dropdown(sports.get_sport_names(), value="basketball", label="Sport")
```

TODO 1: add one more sport

Recall: a dictionary entry is `key: value`, and entries are separated by commas. A list is written in square brackets:

```
pets = {
    "dog": ["sit", "stay", "fetch"],
    "cat": ["scratch post", "laser dot"],
}
```

Your task: add a new sport (tennis, swimming, yoga: your pick) to `SPORTS`, with four example skills.

Hint 1: Copy the shape of an existing entry. Which punctuation character separates entries?

Hint 2: Most students get a `SyntaxError` here because the entry just **above** their new one has no trailing comma.

Think about it (and test it): restart the app. **Does the rest of the app still work with your new sport?** Does the sport appear in the dropdown without you changing `app.py`? Why or why not? What does the photo tip under the dropdown say for your new sport? (Look at the `else` at the bottom of `get_sport_instructions` and say what it did for you.)

> **WHY**
> A good `else` branch is a safety net: a brand-new sport did not crash anything. But "does not crash" is not the same as "works well". Always test the new case yourself.

TODO 2: photo tips for every sport

Camera angle changes every measurement later, so the tips matter. Basketball is finished for you as an example. Read it carefully, because it shows what a good tip contains.

Recall: an `if / elif / else` chain checks conditions from top to bottom and runs only the **first** one that is true:

```
if grade >= 90:
    return "A"
elif grade >= 80:
    return "B"
else:
    return "Keep trying"
```

Your task: replace each `TODO` string with a real tip for soccer, volleyball and dance, and add a branch for the sport you added in TODO 1. Write the wording yourself.

A good tip answers these questions:

- Which **body parts must be in the frame**? (Think about which joints the app will measure later.)
- Which **side** should the camera be on: front, back, or the side of the athlete?
- What **height** should the camera be at?
- Is there a common mistake (athlete too close, cropped feet, dark room) worth mentioning?

Hint 1: For a soccer kick, which leg and foot matter most? For a volleyball set, which part of the body is the focus? For dance, why might the camera need to see the whole body from head to toe?

Hint 2: Compare with `==` (two equals signs) and end every `if` / `elif` line with a colon.

TODO 3: guidance for each skill level

Your task: finish `get_level_guidance` so each of the three levels returns its own sentence. Beginner is written for you. Think about the question in the starter: should a beginner get **one simple tip or five detailed ones**? What about an advanced athlete?

Write your sentences using this checklist:

- **How many tips** should each level get?
- **What tone**: encouraging and jargon-free, or precise and technical?
- Should the advice mention **numbers** (angles) for advanced athletes?

Hint 1: Read the beginner sentence again and copy its *style*: it is written as an instruction ("Give one or two simple...").

Hint 2: The levels should feel like a ladder. Intermediate sits between beginner and advanced.

> **NOTE**
> These sentences are used two ways: `build_suggestions` shows them to the user, and `ai_helper.py` passes them to the AI to control how much it says. If you want them to read well on screen too, reword them so they work for both readers.

> **CHECKPOINT**
> Restart. For each sport, select it and read the photo tip: none should say `TODO`. Then upload a photo, change the level, click Analyze (pose off), and look at the suggestions. In a terminal: `python -c "import sports; print(sports.get_level_guidance('advanced'))"` should print your own sentence.

> **STUCK?**
> `SyntaxError` in `SPORTS` is almost always a missing comma between two entries. `SyntaxError: invalid syntax` on an `elif` line usually means a missing colon.

Step 3: Display measurements and suggestions nicely

File: `app.py` **TODOs 2 and 3**

Goal: the starter prints measurements and suggestions as plain bullet lists. Make the display friendlier.

Read the main handler first

`analyze(...)` is the function Gradio calls when you click **Analyze**. Read it top to bottom and match it to the flow diagram: **validate the upload, optionally detect the pose, measure, build coaching suggestions, optionally ask the AI**. It returns one value for each output component, in the same order as the `outputs=[...]` list at the bottom of the file.

> **CONCEPT**
> `measurements` is a **list of dictionaries**, for example `[{"name": "Right elbow angle", "value": 142.0, "label": "bent"}]` . `value` is `None` when the angle could not be measured. You will create those dictionaries in Step 5. For now, just know their shape.

TODO 2: improve how measurements are displayed

Recall: a function takes inputs, does some work, and **returns** a value. `round(3.7)` gives `4` . An f-string puts values inside text, and `**text**` makes bold text in Markdown:

```
def describe_pet(pet):
    return f"- **{pet['name']}**: about {round(pet['weight'])} kg"
```

Your task: write a function named `format_measurements(measurements)` near the top of `app.py` (outside of `analyze` , no indent) that takes the list of measurement dictionaries and returns **one Markdown string**, one bullet line per measurement. Then make `analyze` call it instead of building the lines inline. Your version should:

- show the angle **rounded** (no long decimals) and with a unit,
- show the label,
- show a **friendly message** (not `n/a`) when `value` is `None` .

Hint 1: Look at the loop that already exists inside `analyze` . You are moving it into a function and improving it.

Hint 2: Build a Python list of line strings inside the function, then join them with `"\n".join(...)` , just like the starter does. Do not forget to `return` the result.

Hint 3: For each dictionary, decide between two cases with `if m["value"] is None:` . Each case produces a different line.

TODO 3: format the suggestions

Recall: `enumerate` gives you a counter along with each item:

```
for number, flavor in enumerate(["mint", "mango"], start=1):
    print(number, flavor)      # 1 mint, then 2 mango
```

Your task: write `format_suggestions(suggestions)` that returns the suggestions as a **numbered** list, and keep the disclaimer (`feedback.DISCLAIMER`) attached to the end. Replace the `suggestions_text = ...`

line in `analyze` with a call to it.

Hint 1: What does the starter line do with `f"– {s}"` ? What do you change to get a number instead of a dash?

Hint 2: The disclaimer must **always** stay visible. Where does the starter put it, and how can your function do the same?

CHECKPOINT

Restart, upload a photo with pose off and click Analyze. The suggestions should be numbered and the italic disclaimer should follow. To test measurements without pose detection, run from inside `off_the_bench` : `python -c "from app import format_measurements as f; print(f([{'name':'Right elbow angle','value':141.6,'label':'bent'},{'name':'Left elbow angle','value':None,'label':'x'}]))"` You should see two lines: the first shows about **142** degrees (rounded) with its label, the second shows your friendly "could not measure" style message.

STUCK?

`NameError: name 'format_measurements' is not defined` means the function is defined below where it is called, or you did not save. If the app breaks on launch, you probably pasted a function inside another function: define both helpers at the top level, with no indent.

Step 4: Detect landmarks with MediaPipe

File: `pose_analysis.py` **Setup and the two practice TODOs**

Goal: turn on pose detection, see the skeleton on a real photo, and write a tiny geometry helper.

What is a landmark?

MediaPipe is a free library from Google. Given a photo, a pre-trained model guesses where **33 body points** are: nose, shoulders, elbows, wrists, hips, knees, ankles and more. You do not train anything. You get coordinates and then use ordinary Python (`if` statements and a little geometry) to measure things.

Each landmark, in the dictionary that `detect_pose` returns, looks like this:

```
"right_elbow": {"x": 0.43, "y": 0.51, "z": -0.12, "visibility": 0.98,
                "x_px": 275.2, "y_px": 244.8}
```

(The numbers here are only an example shape. Yours will differ.)

- `x` and `y` are positions as a **fraction** of the picture (0.0 is left or top, 1.0 is right or bottom).
- `x_px` and `y_px` are the same position in **pixels**. The code uses these for measuring.
- `visibility` says how confident MediaPipe is that the point is really visible (0 to 1).
- "left" and "right" mean the **person's** left and right, not the image's.

> **CONCEPT**
> In pictures, **y grows downward**. A point near the top of the photo has a *small* `y_px` . This surprises almost everyone the first time, and it matters whenever you ask "is the wrist above the shoulder?"

4a. Download the model and check the setup

MediaPipe needs a model file (about 6 MB). From inside `off_the_bench` , with your virtual environment active, run:

```
python pose_analysis.py --download
```

It saves `models/pose_landmarker_lite.task` . Next, ask the file for its own status:

```
python pose_analysis.py
```

If everything is installed, it prints `Pose detection is ready.` If not, it tells you what is missing: either `MediaPipe is not installed (pip install mediapipe). Pose detection is off.` or `Pose model file missing. Run: python pose_analysis.py --download` . The function behind this is `check_pose_setup()` . It never raises an error, so the app can always launch. That is a good habit.

4b. Turn it on

Restart the app. The setup status should now say `Pose detection is ready.` and the checkbox **Detect body landmarks** should be ticked by default (`value=pose_ready`). Upload a clear, whole-body photo and click **Analyze**.

What to expect, if MediaPipe is working: the "Detected landmarks" picture shows red dots and cyan lines drawn over the person, the status says `Pose: Pose detected. Landmark positions are estimates and may be off.` , and a measurement list appears (only one line for now, until Step 5). If the photo has no clear person you will see `No person/pose detected. Try a clearer photo with your whole body visible`

`and good lighting.`

> **STUCK?**
> `Pose model file missing` means you did not run the download command (or ran it in a different folder). If `pip install mediapipe` fails, check `python --version` is between 3.9 and 3.12. A network error during the download usually means you are offline or a school proxy is blocking it.

Read `detect_pose` (no coding)

Read the function in the order it works. It (1) checks the image is a usable color array, (2) checks MediaPipe and the model exist, (3) runs the landmarker on one still photo, (4) converts MediaPipe's output into the simple dictionary above, and (5) draws the skeleton. Its **return value is always a dictionary with the same four keys** (`found` , `status` , `landmarks` , `annotated_image`), even on failure. **Think about it:** why is that useful for the code in `analyze` that reads `pose["status"]` ?

Practice TODO 1: write `get_midpoint`

Recall: a point is a pair `(x, y)` , and you pick out its parts by position, `p[0]` and `p[1]` . The average of two numbers is their sum divided by 2. A function can return two values as a tuple:

```
def combine(dog, cat):
    return (dog[0] + cat[0], dog[1] * cat[1])    # (total age, product of weights)
```

(This is a made-up operation. Your job is a different calculation on the same kind of pair.)

Your task: at the bottom of the helper section of `pose_analysis.py` , write `get_midpoint(p1, p2)` that returns the point exactly halfway between two `(x, y)` points. Could the midpoint of the two hips be useful? (It is a good estimate of the body's center.)

Hint 1: Do the x values and y values separately.

Hint 2: Average x1 and x2, average y1 and y2, return the two results together.

Test it from `off_the_bench` :

```
python -c "from pose_analysis import get_midpoint; print(get_midpoint((0, 0), (10, 20)))"
```

You should see a point that is 5 across and 10 down (it may print as `(5.0, 10.0)`).

Practice TODO 2: look at a real landmark

This one is an experiment, not code to keep. With a test photo (call it `photo.jpg`) in the folder, run:

```
python -c "import numpy as np; from PIL import Image; import pose_analysis as pa; img =
    np.array(Image.open('photo.jpg').convert('RGB')); r = pa.detect_pose(img);
        print(r['status']); print(r['landmarks']['right_wrist'])"
```

Do it twice, once with the wrist **low** and once **raised above your head**. Compare the `y_px` values. Is `y_px` bigger or smaller when the wrist is higher in the picture? Write your answer in your notes, and explain **why** using the Concept box above.

> **CHECKPOINT**
> (1) `python pose_analysis.py` says what is ready or missing. (2) With MediaPipe working, a photo of a standing person shows a skeleton overlay. (3) `get_midpoint((0, 0), (10, 20))` gives the halfway point. If you cannot get MediaPipe installed, do (3) and continue: the next steps can be tested with fake landmarks.

Step 5: Measure body angles

File: `feedback.py` **TODOs 1, 2, 3, 4, 5**

Goal: turn landmarks into **measurements**: a list of joint angles, each with a plain-language label.

The angle math, conceptually

The helper `calculate_angle(a, b, c)` in `pose_analysis.py` is already written. It returns the angle at the middle point `b`, made by the lines from `b` to `a` and from `b` to `c`. For an elbow, `a` is the shoulder, `b` is the elbow, `c` is the wrist.

Inside, each line becomes an **arrow** (a vector). The code finds how much the two arrows point in the same direction and converts that to degrees. You do not need to follow the trigonometry, but you should be able to *read* results:

```
python -c "from pose_analysis import calculate_angle as c; print(c((0, 0), (0, 10), (10, 10)));
    print(c((0, 0), (0, 10), (0, 20)))"
```

```
90.0
180.0
```

Draw the first one on paper. Shoulder at (0, 0), elbow at (0, 10), wrist at (10, 10): the upper arm goes straight down and the forearm goes sideways, a perfect **right angle (90 degrees)**. In the second, shoulder, elbow and wrist all sit on one line, a straight arm, **180 degrees**. A small angle means a tightly bent joint.

> **WARNING**
> The angle is measured **in the 2D photo**. If the arm points toward or away from the camera, the real angle can look very different. This is why the photo tips in Step 2 matter, and why the app is full of disclaimers.

Read the code that is already there

`measure_joint_angle(landmarks, first, middle, last)` asks `get_point` for three landmarks. `get_point` returns `None` when a landmark is missing **or not visible enough** (visibility below 0.5), so `measure_joint_angle` checks `if None in points` and returns `None`. **Think about it:** why must you always check for `None` before using a landmark or an angle?

Now open `analyze_pose` and `classify_elbow_angle`. The starter measures only the right elbow and `classify_elbow_angle` returns a placeholder. The starter numbers the TODOs in a different order than the file. We follow the order that is easiest to write.

Build a fake body so you can test (no photo needed)

Make a file called `fake_landmarks.py` in the `off_the_bench` folder. It builds landmarks by hand, in the same shape as `detect_pose`'s output, so you can test your rules without a photo or MediaPipe:

fake_landmarks.py

```
from feedback import analyze_pose


def lm(x, y, visibility=0.9):
    return {"x": x / 100, "y": y / 100, "z": 0, "visibility": visibility,
            "x_px": x, "y_px": y}


fake = {
    "right_shoulder": lm(0, 0),  "right_elbow": lm(0, 10),  "right_wrist": lm(10, 10),
    "left_shoulder": lm(50, 0),  "left_elbow": lm(50, 10),  "left_wrist": lm(50, 20),
    "right_hip": lm(0, 30),      "right_knee": lm(10, 40),  "right_ankle": lm(0, 40),
    "left_hip": lm(50, 30),      "left_knee": lm(50, 40),   "left_ankle": lm(55, 50),
}

for sport in ["basketball", "soccer", "volleyball"]:
    print(sport)
    for m in analyze_pose(fake, sport):
        print("  ", m["name"], m["value"], m["label"])

fake["right_wrist"] = lm(10, 10, visibility=0.1)      # pretend the wrist is hidden
print(analyze_pose(fake, "basketball")[0])
```

Run it with `python fake_landmarks.py`. Right now it prints one right-elbow line for every sport, with a label that says `not classified yet (finish TODO 3)`. By the end of this step the output will be richer.

TODO 3: classify an elbow angle

Recall: `if / elif / else` checks from the top and stops at the first true condition, so the **order** of your checks matters:

```
if temperature > 30:
    return "hot"
elif temperature > 15:
    return "mild"
else:
    return "cold"
```

Your task: make `classify_elbow_angle(angle)` return a plain-language label for the angle. The starter comment suggests: above about 160 means "nearly straight", about 70 to 160 means "bent", and below 70 means "tightly bent". **Pick thresholds you can defend** and be ready to explain them. Then write a similar `classify_knee_angle(angle)`. Do knees bend the same way elbows do? What would you change?

Hint 1: How many labels do you need? That tells you how many branches there are.

Hint 2: Start with the largest threshold and work down. Why does the order matter, as in the weather example?

Hint 3: You only need two comparisons for three labels, because the final `else` catches everything left over.

TODO 1: find the shoulder, elbow and wrist (right AND left)

Recall: a list can hold tuples, and a tuple can hold a string and a function. Functions are values too, so you can store one and call it later:

```
checks = [("Big", len), ("Loud", str.upper)]
for label, function in checks:
    print(label, function("hello"))
```

Your task: the starter measures only the right arm. Make `analyze_pose` also measure the **left** arm. The left arm's landmark names are `left_shoulder`, `left_elbow`, `left_wrist`. Use the display names `"Right elbow angle"` and `"Left elbow angle"`.

Hint 1: You could copy and paste the right-arm lines and change the names. It works, but what happens when you add knees, shoulders, hips?

Hint 2: Describe each joint **once**, as data, then loop. Try a list where each item is a tuple holding: the display name, the three landmark names, and the function that classifies it.

TODO 2: calculate the angle, and add the knees

Your task: angle calculation is already done for you by `measure_joint_angle`. Add the knees: use the hip, knee and ankle landmarks (`right_hip`, `right_knee`, `right_ankle`, and the left equivalents). Display names: `"Right knee angle"` and `"Left knee angle"`.

Hint 1: Which landmark is the *middle* point (the joint where the angle is measured)?

Hint 2: Treat the knees exactly like the elbows: same kind of data, different landmark names, and the knee classifier you wrote.

TODO 5: choose which joints matter for each sport

Recall: lists can be joined with `+`:

```
print(["dog"] + ["cat", "fish"])     # ['dog', 'cat', 'fish']
```

Your task: use `sport` and `if / elif / else` to pick which joints to measure. A basketball shot cares about the **elbow**, a soccer kick about the **knee**. Choose what makes sense for volleyball, and give the `else` (dance and anything new) a sensible default. For testing with `fake_landmarks.py`, use: basketball = elbows only, soccer = knees only, volleyball = both.

Hint 1: Which of your lists belongs to which sport?

Hint 2: Pick the list *before* the loop, then loop over the chosen list.

TODO 4: handle a hidden landmark gracefully

Your task: for each joint, measure the angle and add a dictionary to `measurements` with the keys `name`, `value` and `label`. If the angle is `None` (a landmark was hidden), the value stays `None` and the label is a friendly sentence. It must **not crash**.

Hint 1: What happens if you pass `None` to your classifier? What does a comparison like `None > 160` do?

Hint 2: Decide with an `if angle is None` check **before** classifying. The two outcomes produce different labels.

Hint 3: The key idea of the loop: the classifier is stored in your data, so `classify(angle)` labels elbows with one function and knees with another, with no extra `if`.

CHECKPOINT

Run `python fake_landmarks.py`. Check these facts (the label words are yours):

- basketball prints two lines: right elbow about **90.0**, left elbow about **180.0**.
- soccer prints two lines: right knee about **45** (it may show as `45.00000000000001`, which is normal rounding noise in computers), left knee about **153.4**.
- volleyball prints all four lines.
- The last line shows the right elbow with value **None** and your friendly label, and the program does not crash.

Each label should make sense for its angle. Does a 180 degree arm get a "straight" type of label?

STUCK?

`TypeError: 'NoneType' object is not callable` or a comparison error involving `None` means you classified an angle that was `None`. `ValueError: too many values to unpack` means a tuple in your list has a different number of items than your loop expects. If every sport measures the same joints, check you used `elif` and compared with `==`.

Step 6: Coaching suggestions

File: `feedback.py` **TODOs 6 and 7**

Goal: turn measurements into advice that depends on the **label** and the athlete's **level**, and add a drill for the sport.

> **CONCEPT**
> This is where facts become opinions. A measurement ("the elbow is 90 degrees in this photo") is a fact. "Try more bend" is an interpretation. Use gentle words: **might**, **try**, **notice**. Avoid **wrong** and **correct**, because one photo cannot know.

TODO 6: a suggestion for each measurement

Recall: a `for` loop visits each item in a list, and `.append()` adds to the end of a list. Dictionary values are read with `d["key"]`:

```
notes = []
for pet in pets:
    if pet["weight"] is not None:
        notes.append(f"{pet['name']} might enjoy a longer walk.")
```

Your task: in `build_suggestions`, loop over `measurements`. For each one that **has a value**, add a suggestion sentence to the list. The sentence should depend on both the measurement's **label** and the athlete's **level**.

Write your sentences using this checklist:

- **Beginners:** gentle, simple, no jargon, maybe no numbers.
- **Advanced athletes:** more specific, can mention the measured angle.
- Always use words like *might*, *try* and *notice*. Never say *wrong* or *correct*.
- Skip joints whose value is `None`: there is nothing to say about something the app could not see.

Hint 1: Writing the sentence logic straight inside the loop gets messy. Consider a small helper function that takes the measurement and the level and returns one sentence.

Hint 2: Inside the helper, think of `level` and `label` as two choices. Which nested or combined `if` structure covers every case without repeating yourself?

Hint 3: Use f-strings to put the joint's `name` and `label` into the sentence. Use `round(m["value"])` only where you decide to show the number.

TODO 7: a drill for each sport (optional)

Recall: `key in dictionary` is `True` when the key exists, and it is the safe way to look something up that might not be there:

```
treats = {"dog": "bone", "cat": "fish"}
if "hamster" in treats:
    print(treats["hamster"])
```

Your task: store one short drill per sport in a dictionary (for example a wall drill for basketball, a cone drill for soccer: your ideas), then add the matching drill to the suggestions. A sport with no drill (like a brand-new one) should simply be skipped, not crash.

Hint 1: Where should the dictionary live: inside the function or at the top of the file? Why?

Hint 2: Which operator checks that a key exists before you use it?

CHECKPOINT

Add these lines to the bottom of `fake_landmarks.py` and run it:

```
from feedback import build_suggestions
for level in ["beginner", "intermediate", "advanced"]: print(level, build_suggestions("basketball", level, analyze_pose(fake, "basketball")))
```

Each level should give **different wording**, the hidden-wrist measurement should get no sentence, and a drill should appear. Then run the real app with a photo and pose on: the suggestion list should include your measured sentences.

STUCK?

`KeyError: 'value'` means a measurement dictionary is missing a key (check `analyze_pose` adds `name`, `value` *and* `label`). A sentence that says "None degrees" means you did not skip `value is None`. Same suggestions for every level means your sentence logic is not using the `level` argument.

Step 7: Optional AI feedback

File: `ai_helper.py` **TODOs 1, 2, 3**

Goal: build a good prompt from your measurements, and display the AI's reply with a permanent warning label.

Read the safety design first

Open `ai_helper.py` and notice three safeguards that are already in the starter. They are not decoration.

- `is_ai_available()` never raises. If there is no key or no `anthropic` package, the app keeps working.
- `get_ai_feedback(...)` only attaches the image `if image is not None and user_consented_to_send_image`. **No consent, no photo.**
- `AI_LABEL` says the output is "not verified biomechanical analysis". It must stay visible.

TODO 1: measurements as a text block

Recall: you can build text from a list by looping, collecting lines, and joining them:

```
lines = []
for pet in pets:
    lines.append(f"- {pet['name']}: {pet['age']} years")
text = "\n".join(lines)
```

Your task: turn the `measurements` list into a readable text block for the prompt, one line per measurement, **skipping any whose value is** `None`. If nothing is left, the block should say that no measurements were available.

Hint 1: Same idea as `format_measurements` in Step 3, but for the AI instead of the screen.

Hint 2: Build the lines list first. Check at the end whether it is empty (an empty list counts as false in an `if`).

TODO 2: a better prompt

Your task: improve the returned prompt. A good prompt states **who the user is, what data we have, what we want back, and what not to claim.** Your prompt must:

- mention the sport and level,
- include your measurement block,
- include the level guidance from `sports.get_level_guidance` (you will need to import it at the top of the file),
- ask for **2 to 3 short tips**,
- tell the AI to **say when it is unsure** and **not to claim certainty** from a single 2D photo.

Hint 1: Importing from another file looks like `from sports import ...` at the top, just like `feedback.py` does.

Hint 2: Build the prompt in pieces with `+=` (add onto the end of a string) so each requirement is its own readable line.

TODO 3: format the reply

Recall: Markdown shows a divider line with `---` and italic text with underscores.

Your task: improve `format_ai_feedback` (a heading, a divider, whatever you like) but **always keep** `AI_LABEL` **visible**, because users must know this is not verified analysis.

Hint 1: Look at how the starter already attaches the label. Keep it.

> **CHECKPOINT**
> You do not need a key for this. Test your prompt directly: `python -c "from ai_helper import build_prompt; print(build_prompt('basketball', 'intermediate', [{'name':'Right elbow angle','value':90.0,'label':'bent'},{'name':'Left elbow angle','value':None,'label':'x'}]))"`
> Read the output as if you were the AI. It should mention basketball, intermediate, the right elbow at about 90 degrees, the level guidance, a request for 2 to 3 tips, and a warning not to claim certainty. It must **not** mention the hidden left elbow.

> **NOTE**
> With no key, ticking "Also ask an AI model" shows `AI feedback is off: no ANTHROPIC_API_KEY found (see .env.example).` followed by the label. That is expected.

Turn it on and run the full test plan

Add your key to `.env` (`ANTHROPIC_API_KEY=...` , no quotes and no spaces), run `pip install anthropic` , and restart. The status line should say `AI feedback is available.`

Test	What you should see
Click Analyze with no photo	"Please upload an image first."
Photo, pose off, AI off	Image size, suggestions, disclaimer
Photo, pose on	Skeleton image, a measurement list, level-aware suggestions
Same photo, change level	Different wording in the suggestions
Tick AI, consent **unticked**	AI answer based on the text measurements only (no photo sent)
Tick AI, consent **ticked**	AI answer that can also describe the photo
Any AI answer	`AI_LABEL` text is shown at the bottom

> **STUCK?**
> `The AI request failed: ...` : read the end of the message. Authentication errors mean a wrong key or a stray space. If it says `no ANTHROPIC_API_KEY found` , `.env` must be in the same folder as `app.py` and you must have restarted. If the answer ignores your photo, check the consent box is ticked.

Step 8: Optional extensions

Finished early? Each of these is a real project. Do them in any order.

1. **Video.** Read frames with OpenCV (`pip install opencv-python`), run `detect_pose` on each one, and record an angle per frame. A movement is a **sequence**, so the angle's change over time says much more than one photo. Plot it.
2. **Compare two photos.** Add a second `gr.Image` , measure both, and report the difference.
3. **Draw the angles.** In `draw_landmarks` , use `ImageDraw` to write each measured angle next to its joint.
4. **Skill drills.** Choose drills from the measurements, not just the sport.
5. **Model sizes.** Try the `full` or `heavy` MediaPipe pose model by changing `MODEL_URL` (and the file name) in `pose_analysis.py` . Compare speed and accuracy.
6. **Use the midpoint.** Use `get_midpoint` for the two hips and shoulders, and measure how much the torso leans.

What you learned

- How an **image becomes a NumPy array**, and why shape is (height, width, channels).
- What **pose landmarks** are, and why `visibility` and `None` checks protect your code.
- **Angle measurement** from three points, and why a 2D photo limits it.
- `if / elif / else` , dictionaries, loops, and storing functions in a list of tuples.
- The difference between **facts** (measurements) and **opinions** (coaching), and why honest labels, disclaimers and **consent** are part of good software.
- Testing code with **fake data** when real input (a photo or an API) is not available.